

State Management in Flutter

State management is the process of **managing and updating data that affects the UI** of an application.

In Flutter, the UI is built using widgets. When data changes, the UI must **rebuild to reflect the new state**.

Example state:

- Counter value
- User login status
- Theme settings
- API data

Basic idea:

None

State Change → Widget Rebuild → Updated UI

Flutter provides multiple state management approaches ranging from **simple to advanced**.

1. Stateless vs Stateful Widgets

1.1 Stateless Widgets

Theory

A **StatelessWidget** is a widget that **does not change during runtime**.

It is immutable and only builds once unless the parent widget rebuilds.

Use StatelessWidget when:

- UI does not change
- Content is static
- No user interaction changes UI

Examples:

- Text

- Icon
 - Static layouts
-

Example: Stateless Widget

```
None
class Greeting extends StatelessWidget {

  @override
  Widget build(BuildContext context) {
    return Text("Welcome to Flutter");
  }
}
```

Here the text never changes.

1.2 Stateful Widgets

Theory

A **StatefulWidget** is used when the UI needs to **change dynamically**.

It consists of two classes:

1. **StatefulWidget**
2. **State class**

The **State** object stores mutable data.

Structure:

```
None
StatefulWidget
  ↓
  State class
    ↓
  build() method
```

Example: Stateful Widget

None

```
class CounterWidget extends StatefulWidget {

  @override
  _CounterWidgetState createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {

  int counter = 0;

  @override
  Widget build(BuildContext context) {
    return Text("Counter: $counter");
  }
}
```

2. Local State using setState()

Theory

`setState()` is the simplest way to update the UI.

When `setState()` is called:

1. State value changes
2. Flutter rebuilds the widget
3. UI updates automatically

Syntax:

None

```
setState(() {
  variable = newValue;
});
```

Example: Counter App

None

```
class CounterPage extends StatefulWidget {
  @override
  _CounterPageState createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {

  int count = 0;

  void increment() {
    setState(() {
      count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Counter")),

      body: Center(
        child: Text(
          "$count",
          style: TextStyle(fontSize: 30),
        ),
      ),

      floatingActionButton: FloatingActionButton(
        onPressed: increment,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

Limitations of setState()

While simple, `setState()` becomes difficult when:

- App grows large
- Multiple widgets share data
- State must be accessed globally

This leads to **advanced state management solutions**.

3. Advanced Technique: Provider (Simplified)

Theory

Provider is one of the **most popular Flutter state management solutions**.

It allows widgets to **access shared state efficiently** without passing data through many widget levels.

Concept:

None

Provider → Provides data

Consumer → Uses data

Advantages:

- Easy to learn
 - Efficient updates
 - Recommended by Flutter team
-

Adding Provider

Add dependency in `pubspec.yaml`.

None

```
dependencies:  
  provider: ^6.0.0
```

Step 1: Create a State Class

None

```
import 'package:flutter/material.dart';  
  
class CounterProvider extends ChangeNotifier {  
  
  int count = 0;  
  
  void increment() {  
    count++;  
    notifyListeners();  
  }  
}
```

`notifyListeners()` tells widgets to rebuild.

Step 2: Provide State

None

```
ChangeNotifierProvider(  
  create: (context) => CounterProvider(),  
  child: MyApp(),  
)
```

Step 3: Use State

None

```
Consumer<CounterProvider>(  
  builder: (context, provider, child) => {  
    // Use provider here  
    return child; // Example return  
  },  
  provider: provider,  
)
```

```
builder: (context, counter, child) {  
  return Text("${counter.count}");  
},  
)
```

4. Riverpod (Basics and Use Cases)

Theory

Riverpod is an **improved version of Provider** created by the same developer.

Advantages:

- Compile-time safety
- No dependency on BuildContext
- Better testing support
- Scalable for large apps

Comparison:

Feature	Provider	Riverpod
Context required	Yes	No
Compile-time safety	No	Yes
Testing	Moderate	Excellent

Example Riverpod Provider

Add dependency:

```
None  
flutter_riverpod: ^2.0.0
```

Create Provider

None

```
final counterProvider = StateProvider<int>((ref) {  
  return 0;  
});
```

Use Provider

None

```
class CounterPage extends ConsumerWidget {  
  
  @override  
  Widget build(BuildContext context, WidgetRef ref) {  
  
    final count = ref.watch(counterProvider);  
  
    return Scaffold(  
      body: Center(  
        child: Text("$count"),  
      ),  
  
      floatingActionButton: FloatingActionButton(  
        onPressed: () {  
          ref.read(counterProvider.notifier).state++;  
        },  
        child: Icon(Icons.add),  
      ),  
    );  
  }  
}
```

5. Overview of BLoC Pattern

Theory

BLoC stands for:

None

Business Logic Component

It separates:

None

UI Layer

Business Logic

Data Layer

Architecture:

None

UI



BLoC



Data

BLoC uses **Streams** and **Events**.

Components:

- Event
- State
- BLoC

Example Flow:

None

User presses button



Event sent to BLoC



BLoC processes logic



New State emitted



UI updates

Advantages:

- Clean architecture
 - Highly scalable
 - Used in enterprise Flutter apps
-

Simple BLoC Example

None

```
class CounterBloc {  
  
  int _counter = 0;  
  
  int get counter => _counter;  
  
  void increment() {  
    _counter++;  
  }  
}
```

In real apps, BLoC uses **Streams** and **StreamController**.

6. Introduction to GetX for State and Route Management

Theory

GetX is a **powerful Flutter package** used for:

1. State management
2. Route management
3. Dependency injection

Advantages:

- Minimal code
- High performance
- NoBuildContext required
- Easy navigation

Install GetX

None

```
get: ^4.6.5
```

Create Controller

None

```
import 'package:get/get.dart';

class CounterController extends GetxController {

  var count = 0.obs;

  void increment() {
    count++;
  }
}
```

`.obs` makes variable reactive.

Use Controller

None

```
final controller = Get.put(CounterController());
```

Reactive UI

None

```
Obx(() => Text("${controller.count}"));
```

Button

None

```
FloatingActionButton(  
  onPressed: controller.increment,  
  child: Icon(Icons.add),  
)
```

Best Practices for State Management

1. Use setState() for Small Apps

Simple UI updates.

2. Use Provider for Medium Apps

Clean and easy to maintain.

3. Use Riverpod for Large Apps

Better testing and scalability.

4. Use BLoC for Enterprise Apps

Strict architecture.

5. Use GetX for Rapid Development

Minimal boilerplate.

Complete Flutter Example (Using setState + Provider)

This example demonstrates:

- ✓ Stateless widget
 - ✓ Stateful widget
 - ✓ setState()
 - ✓ Provider state management
-

Full Flutter Code

None

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: MyApp(),
    ),
  );
}

class CounterProvider extends ChangeNotifier {

  int count = 0;

  void increment() {
    count++;
    notifyListeners();
  }
}

class MyApp extends StatelessWidget {

  @override
```

```
Widget build(BuildContext context) {
  return MaterialApp(
    title: "State Management Demo",
    home: HomePage(),
  );
}

class HomePage extends StatefulWidget {

  @override
  _HomePageState createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {

  int localCounter = 0;

  void increaseLocal() {
    setState(() {
      localCounter++;
    });
  }

  @override
  Widget build(BuildContext context) {

    final providerCounter =
    Provider.of<CounterProvider>(context);

    return Scaffold(
      appBar: AppBar(
        title: Text("Flutter State Management"),
      ),

      body: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
```

```

        Center(
          child: Text(
            "Local State (setState): $localCounter",
            style: TextStyle(fontSize: 20),
          ),
        ),

        ElevatedButton(
          onPressed: increaseLocal,
          child: Text("Increment Local Counter"),
        ),

        SizedBox(height: 40),

        Text(
          "Provider State: ${providerCounter.count}",
          style: TextStyle(fontSize: 20),
        ),

        ElevatedButton(
          onPressed: providerCounter.increment,
          child: Text("Increment Provider Counter"),
        ),
      ],
    ),
  );
}
}

```